Post-Quantum VPN Benchmark Report

Nov 10, 2026 · @Justice

Making our ESP32 VPN post-quantum costs time and memory only once, when a device connects: the handshake takes 219 ms instead of 93 ms. After that, both VPNs perform the same.

What we tested

We tested two versions of the same VPN, using the same code and the same test steps.

	Classical VPN	Post-quantum VPN
Proving who you are	Ed25519	ML-DSA-44 (NIST FIPS 204)
Agreeing on a secret key	X25519	ML-KEM-512 (NIST FIPS 203)
Encrypting messages	AES-256-GCM	AES-256-GCM (same code)
Safe against quantum computers?	No	Yes

The benchmark has four parts:

1. **Part A, on a real ESP32:** an ESP32 at 240 MHz talks to the laptop gateway over a phone hotspot. We time the handshake, the extra delay per message, rekeying, CPU use and memory.
2. **Part B, on the laptop gateway:** many fake ESP32s connect at once, to see how many devices the gateway can handle.
3. **Part C, a fair speed test of the gateway's crypto:** classical and post-quantum are timed in turns, six rounds of 500 runs each, with the laptop plugged in.
4. **Part D, a correctness test:** our post-quantum code is compared with two separate implementations, using 1,000 random inputs.

For times we show the **median** (the middle value) and the **p95** (95% of cases are faster than this). On Wi-Fi a few messages get stuck for a while, so the median is the fair everyday number and the p95 shows the slow cases.

What each measurement means

Each test has a short ID. Tests starting with **B** run on the ESP32, tests starting with **L** run on the laptop gateway.

ID	Measurement (KPI)	What it means in simple words	Why it matters
B0	Known-answer tests passed	Each algorithm is run on fixed inputs and its answer is compared with a published answer	Proves the crypto on the ESP32 is correct, not just fast
B1	AES-256-GCM time per KB	How long the ESP32 takes to encrypt and decrypt 1 KB of data, with no network	Shows the cost of protecting every message
B2	Handshake time	Time from the ESP32's first message until both sides share a key (messages 1 to 4)	How long a device waits before it can send anything
B2	Handshake bytes	Total bytes of the 4 handshake messages	More bytes means more radio time and battery use
B2	Failed handshakes	How many of the 100 handshakes did not finish	Shows if the VPN is reliable
B3	Extra delay (added latency)	Round-trip time of a VPN message minus that of the same message sent without the VPN	The delay the user feels because of the VPN
B4a	Maximum throughput	The most data per second the VPN can deliver, with up to 8 messages on the way	The top speed of the tunnel
B4b	Steady stream delivery	Whether a fixed 102 kbps stream all arrives, and how long replies take	Shows if the VPN keeps up with a sensor that sends all the time

ID	Measurement (KPI)	What it means in simple words	Why it matters
B5	Rekey time	Time to replace the tunnel keys while connected	Rekeying limits damage if a key is ever stolen; it must be quick
B6	CPU use	Percentage of time the ESP32's two cores are busy (our code and Wi-Fi together)	Less CPU means more room for the real application and less power
B7	Heap peak, free heap, unused stack, program size	The most memory used, the memory left, and the size of the program in flash	The ESP32 has only about 300 KB of memory, so this decides if the VPN fits
L0	Known-answer tests on the laptop	The same correctness checks as B0, on the gateway	Proves the gateway's crypto is correct
L1 / Part C	Gateway crypto per handshake	Time the gateway spends on each crypto step of one handshake	Shows how much work each new device costs the gateway
L2	Gateway message crypto	Time to encrypt and decrypt one tunnel message on the laptop	The cost of every message the gateway forwards
L3	Full handshake on the laptop	A whole handshake with a fake device on the same laptop, with no Wi-Fi	The handshake cost without any network delay
L4	Rekey on the laptop	A whole rekey with a fake device on the same laptop	The rekey cost without any network delay
L5	Burst of devices	How many devices get in when 1 to 40 connect at the same moment	Shows what happens after a power cut, when all devices reconnect together
L6	Many open tunnels	Total messages per second, round-trip time and dropped sessions with 1 to 40 tunnels open	Shows how many devices one gateway can serve

ID	Measurement (KPI)	What it means in simple words	Why it matters
Part D	Random cross-check	Our post-quantum code compared with two separate implementations on 1,000 random inputs	Catches bugs that appear only for some inputs

How the numbers are summarised:

- **Median:** the middle value. Half the cases are faster and half are slower. This is the fair everyday number.
- **p95:** 95% of the cases are faster than this. It shows the slow cases.
- **Mean:** the average. One very slow Wi-Fi case can pull it up a lot, so we use the median for anything over Wi-Fi.

Results at a glance

Post-quantum costs more only when a device connects. Everything after that is the same, and rekeying is even faster.

What	Classical	Post-quantum	Difference
Handshake time on the ESP32 (median)	93 ms	219 ms	2.4 times longer
Bytes sent in one handshake	346 B	6,562 B	19 times more
Gateway crypto per handshake (median)	0.18 ms	0.32 ms	1.8 times more
Extra delay per message, 64 B / 1,023 B	+0.7 / +2.3 ms	+0.7 / +2.8 ms	about the same
ESP32 CPU, normal use / 102 kbps stream	1.2% / 9.3%	1.2% / 9.3%	same
Rekey time on the ESP32 (median)	43 ms	25 ms	post-quantum is faster

What	Classical	Post-quantum	Difference
Highest heap use on the ESP32	123 KB	189 KB	about 66 KB more
Program size	1,020 KB	1,031 KB	about 11 KB more
Failed handshakes	0 of 100	0 of 100	same
Correctness tests passed	9 of 9	13 of 13, plus 9,000 of 9,000 random checks	both correct

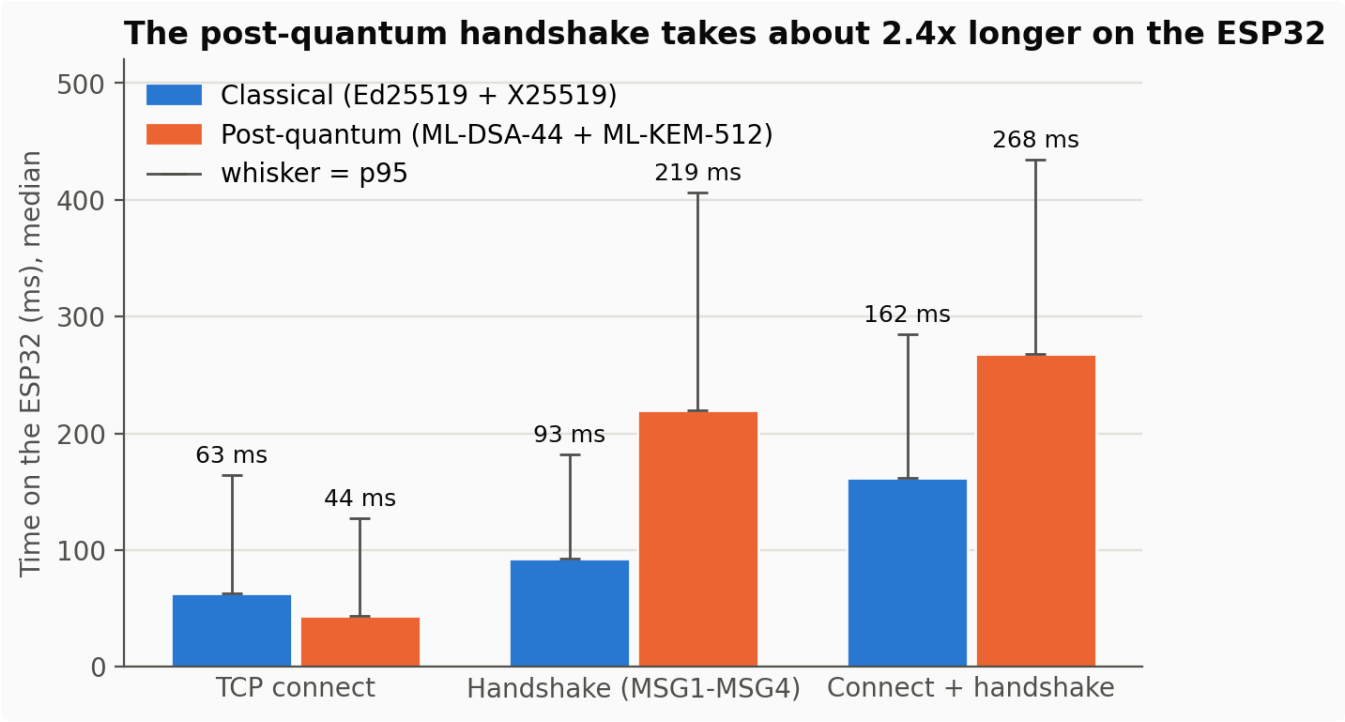
The ESP32 ran at 240 MHz. The laptop gateway was an HP Victus 15, plugged in, in Performance power mode.

1. Handshake time on the ESP32

The post-quantum handshake takes 219 ms instead of 93 ms, which is 2.4 times longer. We timed 100 handshakes for each VPN, and none of them failed.

There are two reasons. First, the ESP32 has to sign with ML-DSA-44, which is slower than Ed25519 and sometimes has to try more than once. Second, the handshake sends 19 times more bytes over Wi-Fi.

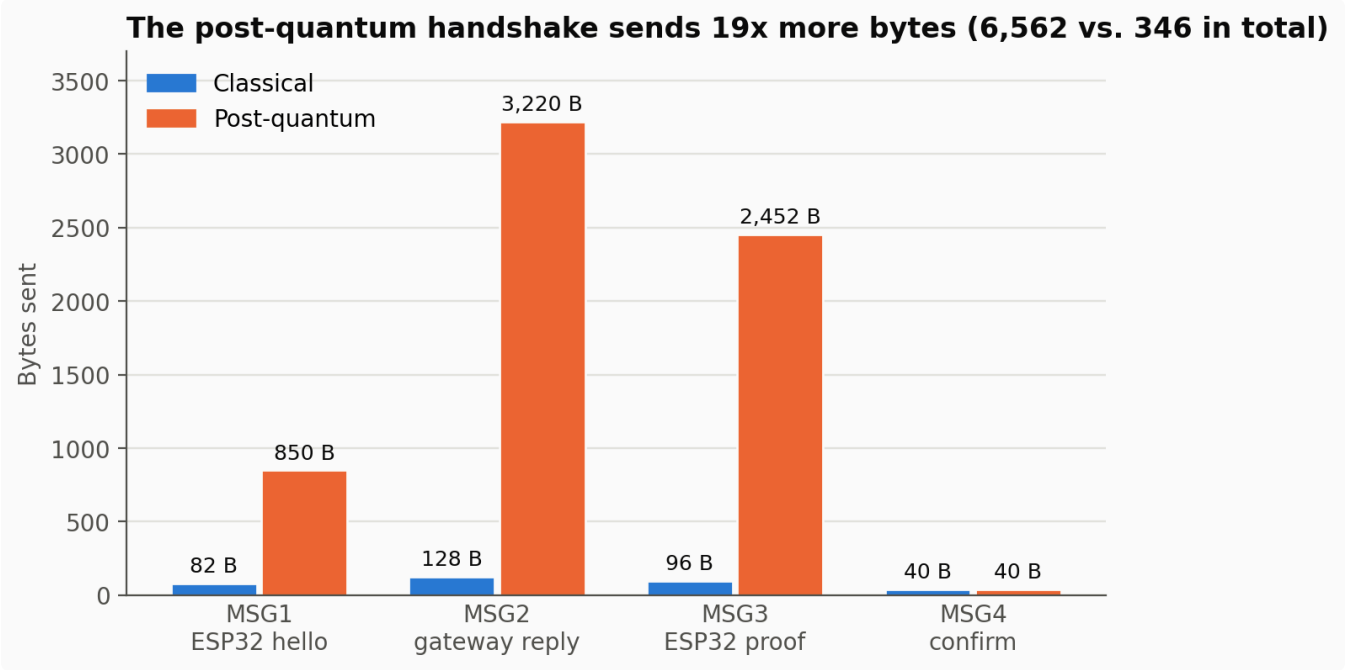
A quarter of a second is fine for an IoT device, because it connects once and then keeps the connection open. The TCP connect time is about the same for both, because it does not use any crypto.



2. Handshake size

The post-quantum handshake sends 6,562 bytes instead of 346 bytes, which is 19 times more.

Post-quantum keys and signatures are much bigger. An ML-DSA-44 public key is 1,312 bytes and a signature is 2,420 bytes, while Ed25519 needs only 32 and 64 bytes. Messages 2 and 3 carry these, so they grow the most. Message 4 is the same 40 bytes in both, because it only confirms the keys.

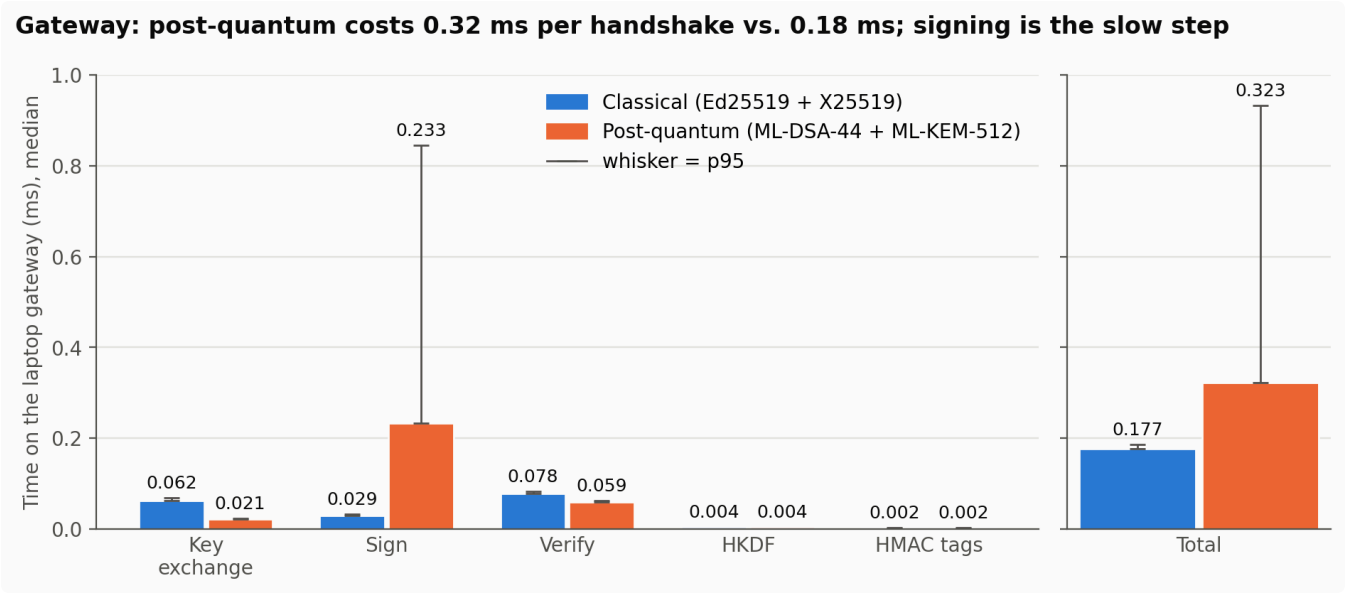


3. Work done by the gateway per handshake

The laptop gateway needs 0.32 ms of crypto per post-quantum handshake, compared with 0.18 ms for classical. Both are tiny: the gateway could do thousands of handshakes per second.

Signing is the only step where post-quantum is slower: 0.233 ms instead of 0.029 ms. Like on the ESP32, ML-DSA-44 signing sometimes has to try again, so its p95 is much higher (0.845 ms). The other steps are equal or faster with post-quantum. ML-KEM-512 is about 3 times faster than X25519, and ML-DSA-44 verifies faster than Ed25519.

We timed classical and post-quantum in turns, so a change in laptop speed would affect both equally. The classical total changed by only 1% between rounds, and the post-quantum total by 15%.

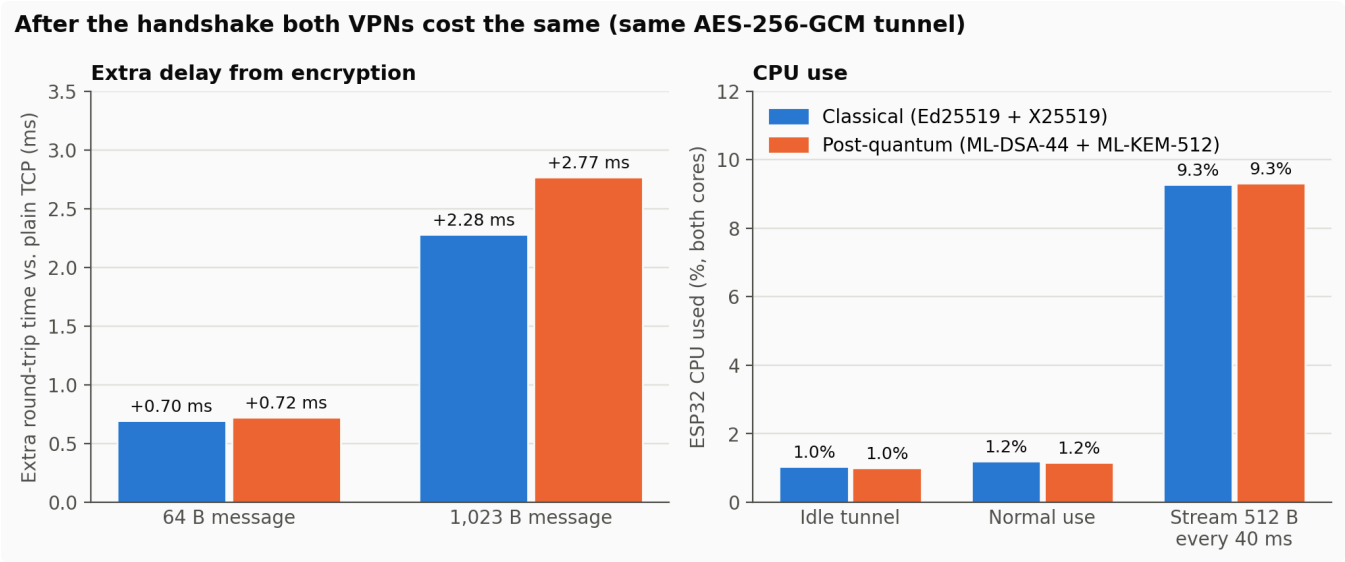


4. After the handshake: delay and CPU

Once connected, both VPNs cost the same, because both encrypt with the same AES-256-GCM code.

- **Extra delay:** the VPN adds less than 1 ms to a small message and about 2-3 ms to a 1,023-byte message, compared with sending it without the VPN. We measured 200 pairs per size, sending a plain message and a VPN message one after the other.
- **CPU use:** the ESP32 is about 1% busy when the tunnel is quiet or in normal use. With a steady stream of 512 bytes every 40 ms (102 kbps), it is about 9% busy. All 750 stream messages arrived in both VPNs.

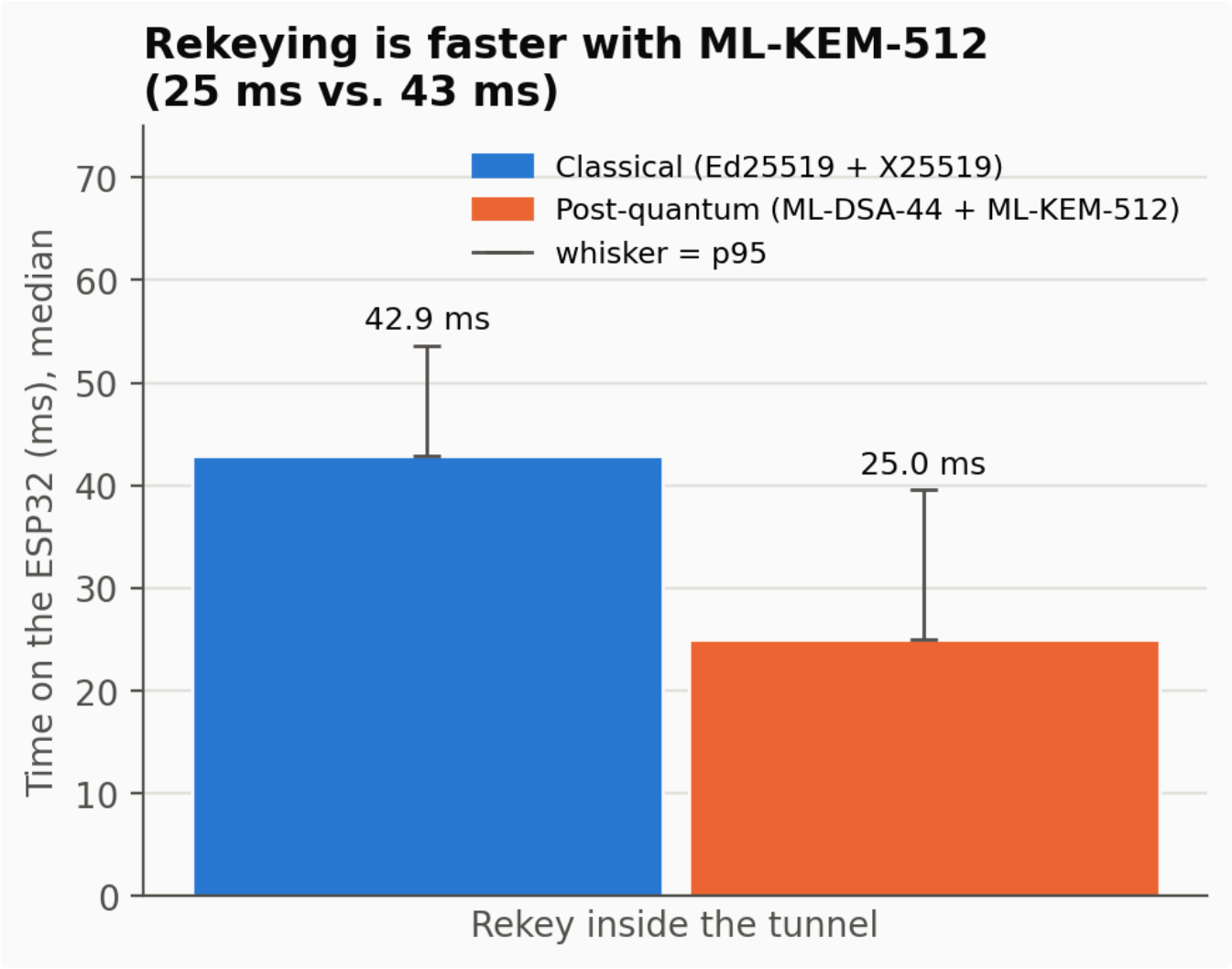
The classical VPN reached 1,119 kbps at full speed and the post-quantum VPN 788 kbps. This difference is most likely the Wi-Fi, not the VPN, because the encryption code and the CPU use were the same. Plain messages with no VPN were also much slower during the post-quantum run: their p95 was 142 ms, against 30 ms in the classical run.



5. Rekeying

Rekeying is faster with post-quantum: 25 ms instead of 43 ms (median of 20 rekeys).

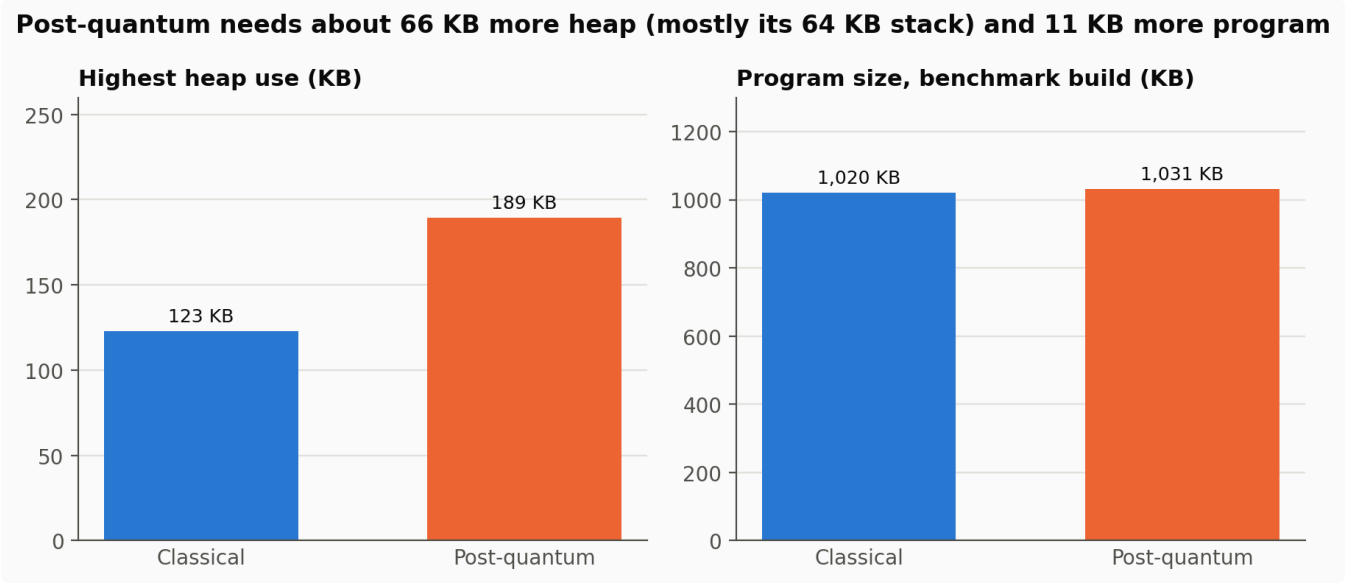
While connected, the VPN replaces its keys regularly: after 1,000 messages or 5 minutes. A rekey runs a new key exchange inside the encrypted tunnel but does not sign anything. ML-KEM-512 is faster than X25519 on the ESP32, so post-quantum wins here. The laptop tests showed the same.



6. Memory on the ESP32

The post-quantum VPN uses about 66 KB more heap memory (189 KB instead of 123 KB) and a program that is about 11 KB bigger.

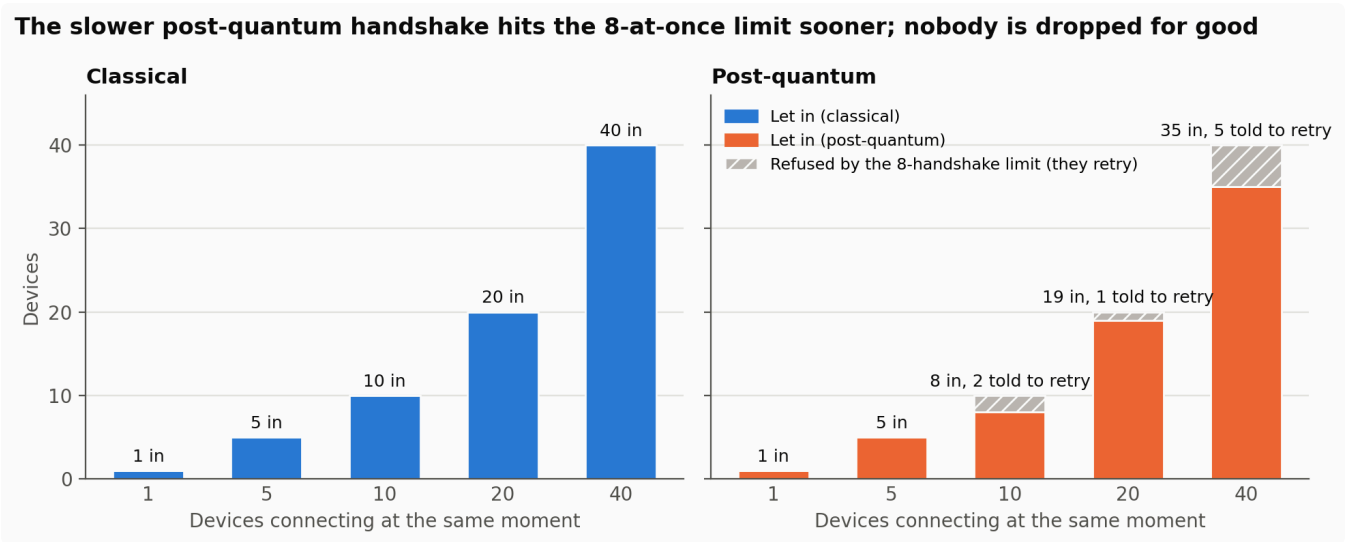
Most of the extra heap is a 64 KB stack for the task that runs the post-quantum code, because ML-DSA-44 signing needs about 34 KB of stack. About 87 KB of heap is still free, so the ESP32 has enough room left for the application.



7. Many devices at once

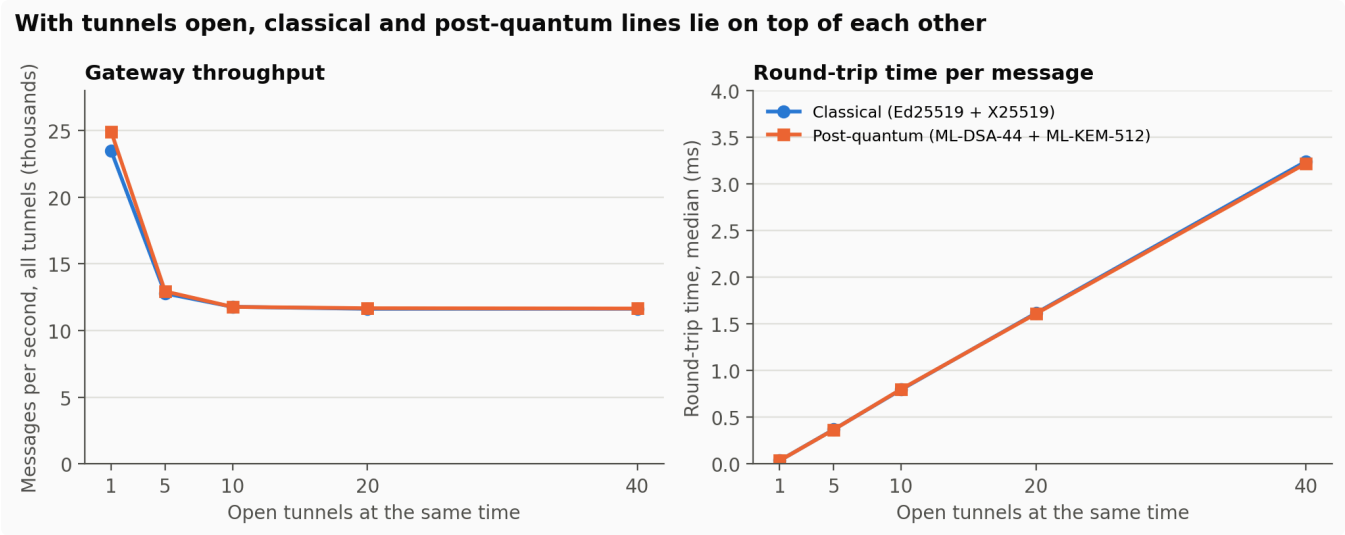
The gateway handled up to 40 devices for both VPNs, and no connected device was ever dropped. These tests used fake ESP32s running as separate programs on the laptop.

Devices connecting at the same moment. To protect itself from attacks, the gateway runs at most 8 handshakes at once. Extra devices are told "busy, try again" and connect on their next try. The post-quantum handshake takes a little longer, so it reaches this limit sooner: with 40 devices, 35 got in on the first try and 5 had to retry. With classical, all 40 got in.



Many tunnels open at the same time. With 1 to 40 tunnels open and sending messages, the two VPNs gave the same results. The gateway handled about 11,600 messages per second in total. The round-trip time grew with the number of tunnels, up to about 3.2 ms

with 40 tunnels.



8. Are the algorithms correct?

Yes. Every algorithm passed its known-answer test on the ESP32, and our post-quantum code matched two separate implementations in 9,000 of 9,000 random checks.

A known-answer test runs an algorithm on fixed inputs and compares the result with a published value. If even one byte is different, something is wrong. This catches mistakes that a self-test would miss, because broken code always agrees with itself.

Algorithm	Checked against	Classical	Post-quantum
SHA-256	FIPS 180-4 example ("abc")	Pass	Pass
HMAC-SHA256	RFC 4231, test case 2	Pass	Pass
AES-256-GCM	NIST GCM test case 16	Pass	Pass
Ed25519	RFC 8032, test 1	Pass	not used
X25519	RFC 7748, vector 1	Pass	not used
ML-KEM-512	kyber-py 1.2.0 (FIPS 203)	not used	Pass
ML-DSA-44	dilithium-py 1.5.1 (FIPS 204)	not used	Pass

One fixed input can miss a mistake that only happens for some inputs. So we also ran 1,000 tests with random inputs. Each test compared keys, ciphertexts, shared secrets and signatures between our code and kyber-py and dilithium-py. All 9,000 checks were

identical.

Full results

Every number below comes straight from the result files (`esp32_results.csv` and `laptop_gateway_results.csv`). Times are shown as median / p95 unless the table says otherwise.

ESP32 results (Part A)

B0. Known-answer tests

	Classical	Post-quantum
Tests passed	9 of 9	13 of 13
Shared tests	SHA-256, HMAC-SHA256, AES-256-GCM encrypt, decrypt and changed-bit check	same
Algorithm tests	Ed25519 make key, sign, verify and changed-signature check; X25519	ML-KEM-512 make key, decapsulate, broken-ciphertext check, round trip; ML-DSA-44 make key, verify reference signature, changed-signature check, round trip

B1. AES-256-GCM on the ESP32, no network (200 runs each)

Message size	Classical: encrypt / decrypt	Post-quantum: encrypt / decrypt	Classical: both, per KB	Post-quantum: both, per KB
64 B	0.074 / 0.073 ms	0.071 / 0.074 ms	2.37 ms	2.35 ms
256 B	0.180 / 0.179 ms	0.177 / 0.180 ms	1.44 ms	1.44 ms
1,023 B	0.606 / 0.606 ms	0.604 / 0.607 ms	1.21 ms	1.21 ms

Small messages cost more per KB because each message has a fixed setup cost.

B2. Handshake (100 handshakes each)

	Classical	Post-quantum
TCP connect	62.7 / 164.5 ms	43.9 / 127.2 ms
Handshake, messages 1-4	92.7 / 181.6 ms	219.3 / 406.4 ms
Connect + handshake	161.9 / 284.9 ms	268.1 / 434.4 ms
Failed handshakes	0 of 100	0 of 100
Bytes sent	346 B	6,562 B

B3. Round-trip time with and without the VPN (200 pairs per size)

	Classical	Post-quantum
64 B without VPN	12.5 / 29.8 ms	11.5 / 142.1 ms
64 B with VPN	13.2 / 23.7 ms	12.3 / 26.1 ms
Extra delay, 64 B (median of the pairs)	+0.70 ms	+0.73 ms
1,023 B without VPN	15.1 / 56.4 ms	13.3 / 150.6 ms
1,023 B with VPN	17.4 / 51.0 ms	16.1 / 37.9 ms
Extra delay, 1,023 B (median of the pairs)	+2.28 ms	+2.77 ms

The p95 without the VPN is sometimes higher than with it. That is Wi-Fi noise, not the VPN making things faster.

B4a. Top speed (30 s, 1,023-byte messages, up to 8 on the way)

	Classical	Post-quantum
Data delivered	1,119 kbps	788 kbps
Messages per second	136.8	96.3
Messages confirmed	4,108	2,890
Rekeys during the test	4	3

B4b. Steady stream (30 s, 512 B every 40 ms)

	Classical	Post-quantum
Sent / delivered	102.4 / 102.4 kbps	102.4 / 102.4 kbps
Messages sent / confirmed	750 / 750	750 / 750
Reply time	14.7 / 84.8 ms	13.9 / 131.4 ms
Rekeys during the test	1	1

B5. Rekey (20 rekeys)

	Classical	Post-quantum
Rekey time	42.9 / 53.6 ms	25.0 / 39.6 ms

B6. CPU use (core 0 / core 1 / average)

	Classical	Post-quantum
Quiet tunnel (20 s)	0.67 / 1.40 / 1.04%	0.64 / 1.35 / 1.00%
Normal use, one ping every 1.5 s (20 s)	0.87 / 1.53 / 1.20%	0.85 / 1.47 / 1.16%
Steady 102 kbps stream (30 s)	5.25 / 13.31 / 9.28%	5.26 / 13.38 / 9.32%

B7. Memory and program size

	Classical	Post-quantum
Total heap	283,500 B	259,348 B
Most heap used	125,848 B	193,816 B
Heap still free	183,024 B	88,732 B
Task stack never used	3,344 B of 8 KB	49,308 B of 64 KB
Program size (benchmark build)	1,044,240 B	1,055,888 B

Repeat check: two separate classical runs

	Run 1	Run 2 (used in this report)
Handshake, messages 1-4	88.7 / 114.5 ms	92.7 / 181.6 ms
AES-256-GCM, 1,023 B, per KB	1.22 ms	1.21 ms
Extra delay, 64 B / 1,023 B	+1.61 / +2.59 ms	+0.70 / +2.28 ms
Top speed	1,208 kbps	1,119 kbps
CPU: quiet / normal / stream	1.02 / 1.17 / 9.11%	1.04 / 1.20 / 9.28%

The two runs agree closely, so the results can be repeated. Run 1 was made before the last two fixes, so its stream reply times and rekey times are not compared.

Laptop gateway results (Part B)

HP Victus 15, plugged in, Performance power mode. The fake devices ran on the same laptop, so there is no Wi-Fi delay here.

L0. Known-answer tests on the laptop

	Classical	Post-quantum
Tests passed	8 of 8	12 of 12

L1. Gateway crypto per handshake (one run of 500)

Step	Classical	Post-quantum
Key exchange (X25519 / ML-KEM-512 encapsulate)	0.063 / 0.070 ms	0.021 / 0.025 ms
Sign (Ed25519 / ML-DSA-44)	0.030 / 0.034 ms	0.230 / 0.838 ms
Verify the device's signature	0.078 / 0.083 ms	0.059 / 0.065 ms
HKDF session keys	0.004 / 0.004 ms	0.004 / 0.006 ms
Two HMAC confirmation tags	0.003 / 0.003 ms	0.003 / 0.003 ms
Total	0.178 / 0.191 ms	0.320 / 0.925 ms

These match Part C (section 3), which repeated the same test in six alternating rounds.

L2. Encrypting one tunnel message on the gateway (2,000 runs each)

Message size	Classical: encrypt / decrypt	Post-quantum: encrypt / decrypt
64 B	0.0064 / 0.0067 ms	0.0064 / 0.0066 ms
256 B	0.0065 / 0.0069 ms	0.0066 / 0.0069 ms
1,023 B	0.0071 / 0.0076 ms	0.0070 / 0.0073 ms

The gateway needs about 7 microseconds per message for both VPNs.

L3 and L4. Full handshake and rekey on the laptop

	Classical	Post-quantum
L3: full handshake, both sides (100 runs)	1.17 / 1.30 ms	2.19 / 3.31 ms
L4: rekey, both sides (20 runs)	0.44 / 0.48 ms	0.15 / 0.35 ms

L5. Devices connecting at the same moment

Devices	Classical: let in / told to retry	Classical: handshake time	Post-quantum: let in / told to retry	Post-quantum: handshake time
1	1 / 0	1.3 ms	1 / 0	1.7 ms
5	5 / 0	6.3 / 6.3 ms	5 / 0	4.7 / 5.5 ms
10	10 / 0	8.8 / 11.4 ms	8 / 2	7.5 / 8.6 ms
20	20 / 0	14.9 / 22.8 ms	19 / 1	14.5 / 21.7 ms
40	40 / 0	29.2 / 47.0 ms	35 / 5	22.5 / 36.1 ms

Handshake times are for the devices that got in. When many arrive together, each waits for the others, so the time grows with the number of devices.

L6. Many tunnels open at the same time

Tunnels	Classical: messages per second	Classical: round trip	Post-quantum: messages per second	Post-quantum: round trip	Dropped (both)
1	23,475	0.036 / 0.059 ms	24,915	0.036 / 0.047 ms	0
5	12,779	0.37 / 0.65 ms	12,932	0.37 / 0.64 ms	0
10	11,778	0.80 / 1.49 ms	11,773	0.80 / 1.50 ms	0
20	11,626	1.62 / 3.09 ms	11,674	1.61 / 3.07 ms	0
40	11,633	3.24 / 6.24 ms	11,651	3.22 / 6.26 ms	0

Messages per second is the total over all tunnels. It drops from 1 to 5 tunnels because the gateway then has to share its time between devices. After that it stays flat at about 11,600.

Part D: random cross-check

	Result
Random tests	1,000 (random seed 2030312455)
Checks identical with kyber-py and dilithium-py	9,000 of 9,000

Where the benchmark code is

The classical and post-quantum projects have the same benchmark files, in `Non_PQC_VPN/` and `PQC_VPN/`. Only the files marked "PQC_VPN only" exist in just one project.

On the ESP32 (folder `esp32_client/` in each project):

File	What it contains	Tests
`SystemBenchmark.cpp` / `.h`	The whole ESP32 benchmark. `runSystemBenchmark()` runs every test in order	B0-B7
`benchKnownAnswer()`	Runs the known-answer tests	B0
`benchLocalAes()`	Times AES-256-GCM with no network	B1

File	What it contains	Tests
`benchHandshake()`	Times 100 handshakes and counts bytes and failures	B2
`benchLatency()`	Plain vs. VPN round trips, one after the other	B3
`benchThroughputMax()`	Top speed with 8 messages on the way	B4a
`benchThroughputFixedRate()`	The steady 102 kbps stream	B4b
`benchRekey()`	Times 20 rekeys	B5
`benchCpuLoad()`, `measureCpu()`	Measures how idle both cores are	B6
`benchMemory()`	Heap, stack and program size	B7
`KnownAnswerTests.cpp` / `.h`	The fixed test inputs and expected answers for each algorithm	B0
`VpnLog.cpp` / `.h`	Turns off the normal printing while measuring, so printing does not slow the timings	all
`config.h`	`BENCHMARK_MODE` (1 = run the benchmark, 0 = normal VPN) and the benchmark settings	all

On the laptop (folder `benchmarking/system_benchmark/` in each project):

File	What it contains	Tests
`gateway_benchmark.py --serve`	Runs the gateway plus a plain echo server for the ESP32 to test against	Part A
`gateway_benchmark.py`	The laptop tests: `l0_known_answer()`, `l1_gateway_crypto()`, `l2_message_crypto()`, `l3_handshake()`, `l4_rekey()`, `l5_burst()`, `l6_many_tunnels()`	L0-L6
`compare_gateway_crypto.py` (PQC_VPN only)	Times classical and post-quantum gateway crypto in turns	Part C

File	What it contains	Tests
`reference_cross_check.py` (PQC_VPN only)	Compares our ML-KEM-512 and ML-DSA-44 with kyber-py and dilithium-py on 1,000 random inputs	Part D
`make_graphs.py` (PQC_VPN only)	Makes the graphs in this report from the result files	graphs

Results (folder `benchmarking/system_benchmark/results/`):

File	What it holds
`esp32_results.csv`	ESP32 results (B0-B7). The classical project also keeps an earlier run in `esp32_results_run1.csv`
`laptop_gateway_results.csv`	Laptop results (L0-L6)
`gateway_crypto_comparison.md` (PQC_VPN only)	Part C results
`reference_cross_check.txt` (PQC_VPN only)	Part D results
`graphs/` (PQC_VPN only)	The 8 graphs in this report

Conclusion

A post-quantum VPN works well on an ESP32. The extra cost is paid once, when a device connects:

- the handshake takes about 0.13 seconds longer (219 ms instead of 93 ms),
- it sends about 6 KB more data,
- it needs about 66 KB more memory.

After connecting, the post-quantum VPN is as fast as the classical one, uses the same CPU, and rekeys faster. For a small IoT device that connects once and stays connected, this is a good price for protection against future quantum computers.